

Day 22 — imputation and custom features inside the pipeline: complete guide

MD Mutasim Billah · 52-week Data Science to ML/AI roadmap · Week 4, Day 2 — definitions, real-world examples, implementations, line-by-line code, and interview prep — paired with `day22_imputation_custom_features.py`

Same complete format as Day 21: for every topic, a definition, a real-world example, the real implementation, and a line-by-line walk-through of what each part of the code does and why. A technical and theoretical interview quiz closes out the guide.

1. The gap Day 21 left open

Definition — Leakage through imputation: the same data-leakage problem Day 21 solved for scaling and encoding, but for missing-value handling: computing a fill value (like a median) from the full dataset before splitting means that fill value was partly informed by test rows.

Real-world example: Every script from Day 10 through Day 21 called `df["Age"].fillna(df["Age"].median())` before ever splitting into train and test. That median is a statistic learned from data — exactly like `StandardScaler`'s mean or `OneHotEncoder`'s category list — and computing it on the full dataset means test rows quietly influenced how missing training values got filled.

```
# The old pattern (Day 10-21) – leakage risk:
df["Age"] = df["Age"].fillna(df["Age"].median()) # median from ALL rows
X_train, X_test, y_train, y_test = train_test_split(df[feature_cols], y, ...)

# Day 22's pattern – split first, impute later, fit only on train:
X_train, X_test, y_train, y_test = train_test_split(df[feature_cols], y, ...)
# X_train and X_test both still contain NaN at this point, on purpose
```

→ in the old pattern, the median is computed once, before the split exists, so it's mathematically influenced by every row including the ones that will become `X_test`

→ in the new pattern, `load_raw()` deliberately does no cleaning at all — missing values are left in place until a fitted imputer decides how to fill them, later, using train data only

2. `SimpleImputer`: a fittable `fillna()`

Definition — `SimpleImputer`: a fittable transformer that learns a fill value (mean, median, most frequent, or a constant) from training data, then applies that same learned value to fill missing entries in any dataset, following the exact fit/transform pattern as every other preprocessing object introduced in Day 21.

Real-world example: It behaves exactly like `StandardScaler`, just for missing values instead of scale: `SimpleImputer(strategy="median").fit(X_train)` computes and stores Age's median from training rows only, and `.transform()` fills NaN with that stored number wherever it's applied — including on `X_test`,

without ever looking at `X_test`'s own values to decide the fill.

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median")
imputer.fit(X_train[["Age"]]) # learns the median from TRAIN only
X_train_filled = imputer.transform(X_train[["Age"]])
X_test_filled = imputer.transform(X_test[["Age"]]) # reuses train's median
```

→ `strategy="median"` — used for numeric columns like Age; `strategy="most_frequent"` is the categorical equivalent, filling with the mode instead of a numeric median

→ exactly like `StandardScaler` in Day 21, `.fit()` only learns and stores a value, while `.transform()` is the step that actually fills anything in

→ calling `.transform()` on `X_test` never recalculates a median from `X_test`'s own Age column — it reuses whatever was learned from `X_train`, which is what keeps this leak-free

3. Nesting a Pipeline inside a ColumnTransformer branch

Definition — Nested Pipeline: a Pipeline used as one of ColumnTransformer's transformers, allowing a single column group to go through more than one sequential preprocessing step (like impute, then scale) instead of being limited to a single transformer per branch.

Real-world example: Numeric columns in this dataset need two things done, in order: fill missing values, then scale. Neither `SimpleImputer` nor `StandardScaler` alone can do both, so the fix is a small two-step Pipeline containing just those two objects — and that whole mini-pipeline is handed to `ColumnTransformer` as if it were a single transformer.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

numeric_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])
```

→ this Pipeline behaves exactly like the outer pipeline from Day 21 — it just happens to be nested one level deeper, used as a single transformer inside a larger structure

→ data flows through it in order: whatever comes out of the imputer step (now free of NaN) becomes the input to the scaler step

→ the step name "imputer" here is what later lets `GridSearchCV`'s nested parameter names reach into this exact sub-pipeline

4. The same pattern for categorical columns

Real-world example: Embarked has a small number of missing values too. The categorical branch mirrors the numeric one exactly: fill first, then transform — just with `SimpleImputer` using the mode

instead of the median, followed by OneHotEncoder instead of StandardScaler.

```
categorical_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore", drop="first")),
])

preprocessor = ColumnTransformer(transformers=[
    ("num", numeric_pipeline, numeric_cols),
    ("cat", categorical_pipeline, categorical_cols),
])
```

- both sub-pipelines are named "imputer" internally, but that's not a conflict — each one only exists inside its own branch ("num" or "cat") of the ColumnTransformer, so their full nested paths are distinct
- the outer ColumnTransformer doesn't know or care that its two branches are themselves multi-step pipelines — from its perspective it's just handing numeric_cols to one transformer-shaped object and categorical_cols to another

5. FunctionTransformer: custom feature engineering as a pipeline step

Definition — FunctionTransformer: wraps any plain Python function into an object with the same `.fit()/transform()` interface as every other pipeline step, letting custom feature engineering logic live inside a pipeline instead of as a separate line of code repeated wherever the data is used.

Real-world example: `FamilySize = SibSp + Parch + 1` has been computed by hand in every single script since Day 13 — nine separate times, copy-pasted with the risk of a typo each time. `FunctionTransformer` turns that one line into a reusable pipeline step that runs automatically, in the same place, every time the pipeline is fit or used to predict.

```
from sklearn.preprocessing import FunctionTransformer

def add_family_size(X):
    X = X.copy()
    X["FamilySize"] = X["SibSp"] + X["Parch"] + 1
    return X

engineer = FunctionTransformer(add_family_size)
```

- `X = X.copy()` — makes a copy before modifying, so the function doesn't silently mutate the original DataFrame the caller passed in, which could cause confusing side effects elsewhere
- `add_family_size` is a completely ordinary function — it doesn't need to import anything special or inherit from a scikit-learn base class to work inside a pipeline
- `FunctionTransformer(add_family_size)` — wraps that function so it exposes `.fit()` (which does nothing here, since this transformation needs no learned statistics) and `.transform()` (which calls `add_family_size` internally)

6. Assembling the full nested pipeline

```

full_pipeline = Pipeline(steps=[
    ("engineer", engineer),
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier(n_estimators=200, random_state=42)),
])

full_pipeline.fit(X_train, y_train)
preds = full_pipeline.predict(X_test)

```

→ three stages, run strictly in order: engineer FamilySize first, since numeric_cols in the preprocessor step already expects a FamilySize column to exist

→ `full_pipeline.fit(X_train, y_train)` — runs FamilySize engineering, then fits and applies imputation/scaling/encoding, then fits the classifier, all from a single call using `X_train` alone

→ `X_train` and `X_test` passed into this pipeline are the truly raw, unmodified DataFrames — missing values, un-encoded categories, and no FamilySize column — with every transformation happening automatically inside the object

7. GridSearchCV over a nested pipeline

Definition — Nested parameter naming: each double underscore in a GridSearchCV parameter name descends one level deeper into a nested Pipeline/ColumnTransformer structure, following the exact chain of step names used when the objects were built.

Real-world example: Reaching the numeric sub-pipeline's imputer strategy from the outermost pipeline requires naming every level in between: the ColumnTransformer step ("preprocessor"), its numeric branch ("num"), and that branch's imputer step ("imputer") — then finally the actual argument ("strategy").

```

param_grid = {
    "preprocessor__num__imputer__strategy": ["median", "mean"],
    "classifier__n_estimators": [100, 200],
    "classifier__max_depth": [4, 8, None],
}

search = GridSearchCV(full_pipeline, param_grid, cv=5, n_jobs=-1)
search.fit(X_train, y_train)

```

→ "preprocessor__num__imputer__strategy" — reads left to right exactly like the nested structure was built: outer ColumnTransformer step, its numeric branch, that branch's imputer step, then the strategy argument itself

→ "classifier__n_estimators" only needs one double underscore, because the classifier step sits directly in the outer pipeline, not nested inside another structure

→ the naming depth always mirrors how deeply the object is nested — there's no limit to how many levels this convention can chain through

Interview prep quiz — technical

Questions about SimpleImputer, FunctionTransformer, and nested pipeline syntax.

Q: What's the difference between `SimpleImputer(strategy='median')` and calling `df['Age'].fillna(df['Age'].median())` directly?

A: `SimpleImputer` is fittable: `.fit()` learns and stores the median from whatever data it's given, and `.transform()` can later reuse that stored value on different data (like a separate test set) without recalculating. Calling `fillna()` with a freshly computed median every time risks that median being computed on the wrong data, such as a combined train+test set.

Q: Why does a nested Pipeline inside a ColumnTransformer branch make sense for numeric columns here, instead of just using SimpleImputer alone?

A: Numeric columns need two sequential operations — fill missing values, then scale. `SimpleImputer` alone can't scale, and `StandardScaler` alone can't fill missing values, so a small two-step Pipeline combining both is passed to `ColumnTransformer` as a single transformer-shaped object.

Q: What does FunctionTransformer actually do to a plain Python function?

A: It wraps the function so it exposes the same `.fit()/transform()` interface every other pipeline step uses, letting an ordinary function like `add_family_size` participate in a Pipeline's steps list without being rewritten as a scikit-learn class.

Q: In the parameter name `preprocessor__num__imputer__strategy`, what does each segment refer to?

A: 'preprocessor' is the `ColumnTransformer` step's name in the outer pipeline, 'num' is the name given to the numeric branch inside that `ColumnTransformer`, 'imputer' is that branch's `SimpleImputer` step name, and 'strategy' is the actual constructor argument being tuned.

Q: Why does `add_family_size()` call `X.copy()` before modifying the DataFrame?

A: To avoid mutating the original DataFrame object passed into the function. Without the copy, adding a column would modify the caller's original data in place, which could cause unexpected side effects anywhere else that same DataFrame is used.

Q: Why must the 'engineer' step come before the 'preprocessor' step in the pipeline's steps list?

A: The preprocessor's `numeric_cols` list includes 'FamilySize', which doesn't exist until the engineer step creates it. If the order were reversed, the preprocessor would fail looking for a column that hasn't been added yet.

Interview prep quiz — theoretical / conceptual

Questions about why nested pipelines and fittable imputation matter in practice.

Q: Why is imputation considered a form of data leakage risk if not handled carefully, the same way scaling and encoding are?

A: Any statistic learned from data — a mean, a median, a mode, a category vocabulary — can leak test-set information into training if it's computed before the train/test split. Imputation is easy to overlook because `fillna()` feels like simple data cleaning rather than a 'model-adjacent' step, but the underlying risk is identical.

Q: What's the advantage of moving feature engineering logic (like `FamilySize`) into a `FunctionTransformer` instead of leaving it as a line of code before the pipeline?

A: It guarantees the exact same transformation runs identically every time the pipeline is used — during training, during cross-validation folds, and on new data at prediction time — instead of relying on a developer to remember to copy that line correctly everywhere the data is touched.

Q: Why does nesting pipelines inside a `ColumnTransformer` scale better than writing a long manual sequence of fit/transform calls for each column group?

A: The nested structure documents its own logic — reading the `Pipeline` and `ColumnTransformer` definitions tells you exactly what happens to each column group, in what order. A long manual sequence of separate fit/transform calls has no such structure and is easy to get subtly wrong or apply inconsistently between training and prediction.

Q: Why might a data scientist choose `strategy='median'` over `strategy='mean'` for imputing a column like `Age`, and how does that choice interact with `GridSearchCV` here?

A: Median is more robust to skew and outliers, which matters for a right-skewed variable like `Age`. Rather than assuming median is always the right choice, including it as a tunable parameter in the grid (`preprocessor__num__imputer__strategy`) lets cross-validation empirically confirm which strategy actually performs better on this dataset.